Recommendation: Adopt a local-first, two-engine data architecture in stages. Keep SQLite as the durable transactional source of truth for users, portfolios, positions, watchlists, alerts, ingestion state, and operational metadata. Introduce DuckDB as a local analytical engine for large historical scans, feature generation, research aggregates, and ML training datasets only after a representative benchmark. Treat Parquet files as reproducible analytical snapshots, not a second authority.

Local Data Architecture Review & Recommendation

Decision area	Recommendation
Operational truth	SQLite remains the only writer/authority for operational and user-owned records.
Analytics and ML	DuckDB reads curated local snapshots and computes broad scans/features; no direct portfolio writes.
Data files	Parquet partitions for rebuildable, versioned analytical snapshots where measured beneficial.
User experience	Keep Python + plain JavaScript + local app process; improve query plans, summaries, cache, and provenance before framework change.
Deployment	No mandatory database server or network dependency; local startup and offline access stay first-class.

What this report does not assume

- Moving to DuckDB or another engine does not fix inaccurate source data, bad units, stale feeds, synthetic values, survivorship bias, or incorrect formulas. Those need explicit validation and provenance.
- A local-first application still fetches exchange/provider data when connected. “Local” describes where the application and durable working data run; it does not mean the external feeds themselves are local.
- The repository contains prior performance targets and proposals, but those are not before/after results. No latency or speedup percentage is claimed here.

Decision status: approve an instrumented benchmark and a reversible analytical pilot; do not schedule a full database replacement until the benchmark establishes a clear win.

PREPARED FOR PRODUCT ARCHITECTURE DECISION

29 October 2026 | Repository and local database review

Decision focus: local read/write speed, analytics throughput, and data correctness

1. Scope and evidence

This assessment uses the checked-out source tree, README and dependency declarations, current local SQLite catalog and row counts, application connection helper, overview design audit, and container configuration. Database catalog and row counts were read through a read-only SQLite connection. A full PRAGMA quick_check was not completed during this review, so this report does not certify physical integrity.

Evidence	Observed state	Architecture implication
Primary runtime	Python 3.11+ backend and plain JavaScript/PWA UI; app.py contains a large route/handler surface.	Avoid simultaneous framework rewrite; isolate data access behind repository/service functions.
Persistent DB	market_data_v2.db is 1,537,445,888 bytes (~1.54 GB decimal / 1.43 GiB); SQLite journal reports WAL.	Single local file is simple and offline-friendly; mixed workloads and backup/WAL discipline matter.
SQLite catalog	40+ application tables include market, options, news, ML/signals, portfolio, auth and cache domains.	Logical data ownership and access policies are not explicit enough; split workload boundaries.
Largest counted tables	intraday_prices 3,717,897; daily_prices 2,087,508; option_chain_data 818,123; volume_anomalies 300,684; bulk_block_deals 216,414.	Large range scans compete with transactional reads/writes inside one database.
Code usage	App and ingestion scripts open sqlite3 directly. app.py _db() sets WAL, synchronous=NORMAL, 8 MB cache per connection, busy_timeout 30s.	Repeated PRAGMA policy and scattered SQL make tuning and correctness harder; centralize boundaries.
Analytics engine	duckdb is declared in requirements.txt and README describes hybrid use, but source search found no DuckDB application usage.	Current docs overstate deployed hybrid. Treat DuckDB as optional/proposed, not operational fact.
Vector/model artifacts	chroma_db/ exists; Chroma and embedding dependencies are optional. models/ and pickle-based training code exist.	Keep vector search/model artifact lifecycle separate from relational and columnar price data.
External fetches	NSE archives/APIs, Yahoo Finance, Google News/RSS and other network clients appear in ingestion/runtime paths.	Local storage cannot remove provider latency, entitlements, feed delays, or availability.

Footprint arithmetic: 1,537,445,888 bytes is approximately 1.54 GB decimal or 1.43 GiB. SQLite page size is 4,096 bytes; catalog reported 375,353 pages, no free pages, user_version 0, cache_size -2,000, mmap_size 0. These are observed settings, not performance measurements.

Current system boundaries

The persistent database mixes transactional state (users, portfolios, positions, alerts), time series (daily and intraday prices), high-cardinality market snapshots (option chains), derived research (signals, anomalies, pivots, summaries), document/news records, and caches. Several scripts ingest directly into this file, app.py executes direct SQLite queries, and an optional Chroma vector store exists alongside model files. This creates operational coupling even though all components run locally.

Data path summary

External providers / exchange archives → Python download and scheduler scripts → SQLite tables and local files → app routes and analysis/ML modules → plain-JS UI. Some runtime routes also perform provider calls. ML training reads local prices and writes model artifacts. A Chroma-backed RAG path is optional. The proposed design moves expensive bulk analysis into a separated local analytical path.

2. Current-state assessment

Strengths to preserve

- SQLite is embedded, portable, serverless and appropriate for a single-user local application with modest transactional concurrency. It supports offline operation and straightforward backup/restore.
- WAL is active in the inspected database and allows concurrent readers with a writer under SQLite's documented model.
- Targeted composite indexes exist for common symbol/date access paths, including daily_prices, intraday_prices, option-chain uniqueness, signals and positions.
- RAM caching and asynchronous background ML startup already keep some work off the immediate request path.
- DuckDB and local vector-store dependencies appear in the project, so analytical separation can be piloted without adopting a network service.

Risks and gaps

- A single large SQLite file combines interactive transactions with analytical scans and bulk ingestion. WAL helps reader/writer overlap, but SQLite still serializes writes; long readers can delay checkpoints and grow WAL sidecars.
- Connection setup is repeated across scripts and route code. The shared helper changes journal mode at connection creation and sets a small per-connection cache; each connection has its own cache. Tune centrally and by workload.
- Database user_version is 0. Add a versioned migration ledger and explicit backup-before-schema-change process before structural evolution.
- Raw vs normalized vs derived ownership, source provenance, observation time, session/time zone, unit, algorithm/model version, and correction policy need consistent data contracts.
- Local storage cannot guarantee fresh market data: providers may be delayed, incomplete, rate-limited or change behavior. Store source and observed-at metadata, then display freshness honestly.
- Data correctness is independent of query speed. Existing audit notes identify formula-generated derivative values and synthetic intraday points in UI flows. Faster queries can make incorrect answers arrive sooner.
- README describes DuckDB as though it serves tick data, but source search found no DuckDB application usage. Correct docs before stakeholders rely on it.

Correctness before speed

Calculations become more correct through explicit types, units, constraints, deterministic transformations and validation rules, not through a database brand. Store raw observations with immutable provenance, normalize exchange symbols/contracts/time zones, use money-safe numeric representation where required, define missing versus zero, and record derivation versions. ML datasets need point-in-time inputs and labels to prevent leakage.

3. Target local-first architecture

Recommended topology

External sources: NSE/BSE/MCX archives, provider APIs, news
Ingestion adapters: retries, rate limits, source timestamps, raw payload hashes, schema validation
SQLite operational store: users, portfolios, positions, watchlists, alerts, job state, normalized latest state
Local analytical lane: curated Parquet snapshots + DuckDB query/feature jobs
Derived snapshots, backtests, ML features/training sets, research aggregates; versioned and reproducible
Python application services: repository boundaries, calculation services, cache, provenance-aware APIs
Local web UI: plain JS/PWA, compact first-load payload, stale/error/unavailable states

Ownership rules

Component	Owns	Must not own
SQLite	Authoritative user/trading state, ingestion checkpoints, curated operational reference entities, small latest-value snapshots.	Large repeated scans, feature matrices, regenerable analytics copies, model binaries.
DuckDB	Local analytical queries over immutable/versioned Parquet and selected curated exports; backtest/feature computations.	Concurrent application writes, portfolio/order source of truth, mutable multi-process serving state.
Parquet snapshots	Rebuildable, partitioned, versioned outputs from validated sources; stable analytical handoff.	Unversioned hand edits or independent business truth.
Chroma / models	Text retrieval and model binaries/metadata with versions and rebuild inputs.	Primary numeric time-series store or authoritative financial metrics.

Why this split fits the app

- SQLite remains the low-complexity option for local transactions; no daemon, credentials or network are needed to open the application.
- DuckDB is designed for local analytical scans, vectorized execution and columnar data; route large historical scans, joins, feature generation and backtests there after measurement.
- Parquet can reduce repeated extraction and provide compressed, typed, reproducible snapshots. Partition by dataset/time where useful and avoid tiny files.
- Separate read/compute from authoritative writes. Derived output returns through controlled migration/writer interfaces or rebuildable files; analytics never mutate portfolio truth.
- A single-user local installation remains one-click and self-contained. No PostgreSQL, Redis or always-on service is required.

4. Alternatives considered

Option	Advantages	Costs / limits	Decision
SQLite only, tuned	Simple install; transactional integrity; strong indexed lookups; no service.	Large analytical scans share cache/I/O and writer domain; still need materialization and workload control.	Keep as OLTP core; profile and tune.
DuckDB only	Strong local columnar analytics; efficient scans and aggregates.	Not preferred owner for mutable, concurrent OLTP; migration of auth/portfolio/tracker writes adds risk.	Reject wholesale replacement; use for isolated analytics.
SQLite + DuckDB + Parquet	Preserves transactional app; isolates analysis; portable/offline; reversible; supports reproducible ML data.	Needs snapshot orchestration, freshness rules, storage budget and observability.	Recommended target, phased behind measured benefit.
PostgreSQL local service	Concurrent writers, constraints and mature server operation.	Install/upgrade/backup/port/security; extra lifecycle; analytics still needs design.	Defer unless multi-user, remote clients or shared writes become requirements.
Redis/cache service	Fast shared ephemeral cache across processes.	Extra service; invalidation/staleness risks; does not fix source quality or analytics.	Defer until contention is measured.
NoSQL/document DB	Flexible document payload storage.	Does not naturally solve trading transactions or numeric analytical scans; adds duplication.	Do not select as core DB.

Why not “everything in DuckDB”

The app includes authentication, portfolios/positions, trackers, scheduler checkpoints and ingestion logs. These are correctness-sensitive row-level operations. Replacing SQLite broadly introduces migration and operational work without evidence that transactional lookups are slow. Use each engine for the workload it handles well.

Why not PostgreSQL now

PostgreSQL is justified by concurrent users/writers, remote clients, shared multi-machine state, or requirements SQLite cannot meet. Those are not established in the local-running goal. A service database adds lifecycle burden and does not eliminate the separate need for analytical pipelines. If requirements change, PostgreSQL is the leading OLTP migration candidate.

5. Expected outcomes and limits

Goal	Mechanism	Expected direction	Limit / proof required
Faster local reads	Indexes, query plans, bounded payloads, read-only connections, summary snapshots; analytics to DuckDB.	Lower page p95 and less interference from history scans.	Benchmark same query/data, cold and warm; no speedup percentage promised.
Faster writes	Short transactions, batched upserts, staged imports, avoid writer overlap.	Less lock waiting and per-row overhead.	SQLite still has one writer at a time; DuckDB does not remove this.
Faster ML prep	Columnar historical scans, date/symbol-pruned Parquet, batch feature computation.	Faster wide scans and repeated feature rebuilds.	Training may be dominated by Python/model fit, CPU/RAM or transfer; measure end-to-end.
Faster pages	Small initial request, compact summaries, lazy panels, cache with invalidation.	Avoid synchronous full-universe analytics on first load.	DB tuning cannot reduce provider latency or rendering cost.
Correct calculations	Typed schemas, units, null semantics, provenance, point-in-time data, formula services, quality gates.	Reject invalid/stale/mismatched input before metrics or ML.	Needs domain rules, validation and source entitlements; engine alone does nothing.
Reliable local use	Embedded deployment, explicit migrations/backups, local snapshots.	Fewer services; offline access to stored data.	Refresh, news, initial provisioning and optional cloud sync need network.

What “local for the life of the app” should mean

- Application, operational store, analytical store, models, caches and user state live on the user machine or chosen local disk.
- App opens and calculates from stored data without a cloud DB. Network refresh is optional and reports unavailable when offline.
- Cloud sync scripts should be opt-in and never silently overwrite newer local DB state. If sync stays supported, use versioned exports or a snapshot protocol instead of copying an actively changing SQLite file.
- Backups include a transactionally consistent SQLite backup plus versioned analytical snapshots and model metadata. WAL sidecars are runtime state; use SQLite backup API or coordinated checkpoint/close.

Read/write policy detail

Use read-only connections for request reads, with shared busy timeout, row factory and bounded query time. Keep write transactions short; batch rows; never perform network fetches inside a transaction; serialize ingestion commits. Set journal mode during initialization/migration rather than every connection. Tune page cache and mmap only after measuring RAM and working set. Observe WAL checkpoint progress instead of blind aggressive checkpointing.

6. Migration and delivery roadmap

Phase 0 - Baseline and guardrails (no migration)

- Define reference hardware, representative DB copy, UI/ingestion/ML workload; record CPU/RAM/storage, Python/SQLite versions, row counts and market date.
- Instrument route, SQL, serialization, provider fetch, feature build and model fit separately. Capture p50/p95/p99, bytes, peak RSS, lock/busy counts, WAL bytes and query plans.
- Profile hot queries with EXPLAIN QUERY PLAN. Remove duplicate indexes only with evidence; add indexes only for measured plans.
- Capture correctness baseline: source, observed time, session, units, nulls, formula version, duplicates, row counts and representative result checksums.
- Add schema migration ledger and verified backup/restore before schema changes.

Phase 1 - Optimize current local path

- Centralize SQLite connection roles and PRAGMA policy. Use read-only URI connections for request reads and prevent accidental route writes.
- Batch/stage ingestion with idempotency keys and domain uniqueness. Avoid request-time broad scans.
- Precompute compact overview aggregates after ingestion. Key cache and snapshots by data version/freshness for deterministic invalidation.
- Move formulas into tested calculation services with explicit units and missing-data semantics. Show unavailable rather than fabricated output.
- Correct docs: mark DuckDB and Chroma optional/proposed until code paths use them.

Phase 2 - DuckDB pilot (read-only analytics)

- Choose one costly repeatable workload: multi-symbol feature prep, cross-sectional scans or backtest dataset construction.
- Export a bounded, validated SQLite snapshot to Parquet. Include snapshot time, source DB revision/counts, schema version, symbol/date bounds and transformation version.
- Run identical computations on existing and DuckDB paths. Compare exact outputs or documented floating-point tolerances, missing data and leakage behavior.
- Measure end-to-end time including export, CPU, peak memory, disk I/O, temp spill, freshness lag and concurrent UI/ingestion impact.
- Adopt only if improvement exceeds a pre-agreed threshold and correctness is equal or better; retain rollback for several production-like cycles.

Phase 3 - Productionize proven work

Migration safety

Do not convert or replace the live 1.54 GB database in place. Use verified backup, immutable source, shadow export, deterministic counts/checksums and explicit rollback switch. No data deletion, retention reduction or cloud-sync change is part of this recommendation.

- Publish atomically: write temporary partition, validate schema/counts/checksums, then publish manifest/current pointer. Failed work leaves last-good snapshot available.
- Record feature version, label horizon, cutoff, source snapshot, code version, seed and training window for ML outputs.

- Bound DuckDB connections/workers; monitor memory, temp files and long scans; avoid unconstrained concurrent jobs on local disk.
- Define retention/rebuild before keeping multiple full copies. Provide one-command rebuild and cleanup of regenerable artifacts.
- Expose store versions, snapshot age and last validation in startup diagnostics.

7. Benchmark plan and decision gates

A benchmark is required before saying “faster.” Test the current SQLite path against each proposed optimization separately, then test their combination to avoid misattribution.

Workload	Case	Measure
Overview UI	30 cold + 100 warm loads, same browser/DB/session/viewport.	First useful content, API p50/p95/p99, request count, payload bytes, DB vs total time.
Historical reads	Recent symbol, one-year/full-history range, cross-sectional date scan.	Wall time, plan, pages read, CPU, rows, peak RSS.
Ingestion writes	Daily batch plus 10 concurrent UI readers.	Rows/sec, transaction duration, lock waits, request tail latency, WAL growth/checkpoint.
ML prep	Identical feature build and training data, code/model/hardware.	Export+transform+fit time, CPU/RAM, output parity and leakage checks.
Backtest/research	Representative multi-symbol join and repeated trials.	Warm/cold runtime, temp disk, result parity, reproducibility, total footprint.
Recovery	Restore backup and rebuild analytical snapshot.	Recovery point/time, counts/checksums, app startup, validation.

Decision gates

Gate	Pass condition
Correctness	No unexplained result differences; provenance, units, null/session semantics preserved; leakage checks pass.
Performance	Proposed pilot gate: at least 2x faster for selected analytical job after export cost, while UI/ingestion p95 does not worsen by >10%. Proposed, not achieved.
Resources	Peak RAM and disk fit documented reference-machine budget; no unbounded spill or duplicate history without retention agreement.
Reliability	Atomic snapshots; failed job preserves last-good; restore/rebuild succeeds.
Simplicity	One-click local install/start; no required network service; logs expose stale snapshot and refresh failure.

Thresholds are proposed acceptance criteria. Adjust them to target hardware and user-perceived goals before the pilot starts.

8. Risks, controls, and open product inputs

Risk	Control
Two copies diverge	One authority per domain; snapshots versioned/rebuildable; manifests tie to source revision and counts.
Snapshot stale	Show as-of and refresh state; define dataset freshness SLA; fail closed for current-state claims.
Disk/RAM pressure	Measure RSS/temp spill; partition selectively; reserve storage; retention and cleanup; cap workers.
SQLite contention persists	Short transactions, batching, serialized ingestion, checkpoint monitoring; PostgreSQL only if multi-writer need exists.
Migration loses data	SQLite backup API, immutable source, counts/checksums, restore test, config switch and rollback.
Performance harms correctness	Parity and domain invariant checks, date/session/unit checks and point-in-time ML leakage review.
Provider failure/policy limits	Persist only authorized data; record source/fetch time; separate external feed SLA from local query metrics.
Cloud sync conflicts with local ownership	Opt-in versioned sync; never replace a running DB file from remote storage.

Product questions for sizing

- Target PC specification and expected max database size / years of intraday retention.
- Will multiple app processes/users or another device write the same data? That changes transaction requirements.
- Which feeds are licensed/authoritative for equities, derivatives, commodities, indices and news; what are latency and retention rights?
- Acceptable refresh age for each data class, and offline expectations after initial provisioning.
- Disk budget for snapshots and model experiments, plus acceptable backup/restore time.

Answers are not required for the direction here. They are needed before locking hardware targets, retention, feed SLAs and acceptance thresholds.

Source notes

- Repository reviewed: README.md, requirements.txt, app.py, ingestion scripts, Dockerfile, docker-compose.yml, and design_audit/OVERVIEW_PAGE_IMPROVEMENT_PROPOSAL.md.
- Local DB catalog and row counts inspected read-only on 29 Oct 2026. File size 1,537,445,888 bytes; journal_mode=wal. PRAGMA quick_check did not complete.
- Technology claims reflect repository evidence and design judgment; no provider entitlement review or performance benchmark was performed.

Final recommendation

Keep SQLite as the operational backbone. Measure and tune it. Move only proven large analytical and ML preparation workloads into a local DuckDB + Parquet lane. Keep the application, data, and user state locally usable; make refresh optional, versioned and transparent. Establish correctness and performance baselines before migrating data.